一、这个项目是什么

Suzza 是一个用 **C++20 + Vulkan** 写的**小型 3D 渲染引擎**，约 70 个源文件、1 万行左右。它包含四个核心特点：

1. **Actor + Component** 实体组件架构（类似 Unity/UE 的思路，但极其简化）
2. **自己写的简易反射系统**：用 clang 注解 + 外部 LLVM 工具自动生成元数据代码
3. **Vulkan 渲染后端**：手动实现交换链、管线、命令缓冲、bindless 纹理等
4. **ImGui 编辑器**：运行时显示 Hierarchy 树和 Inspector 属性面板

它适合用来理解"一个游戏引擎是怎样由 0 到 1 组合起来的"，不适合做商业项目。

二、先跑起来：能运行才读得懂

依赖：MSVC（项目只验证过 MSVC）、CMake 3.8+、Vulkan SDK、C++20。

推荐用 Ninja 预设（CMakePresets.json 里的 x64-debug），在 *x64 Native Tools Command Prompt for VS* 里执行：

bat

```
cmake --preset x64-debug

cmake --build --preset x64-debug
```

为什么必须先把工程配置成功：反射代码生成器 tool/ReflectGenerated/GENERATED_REFLECT_CODE_PROGRAM.exe 依赖 build/compile_commands.json，而只有 Ninja/Makefile 生成器才会产出这个文件（见 CMakeLists.txt:112-135）。所以不要直接使用 cmake -G "Visual Studio 17"，否则反射生成步骤会缺少输入。

编译时会自动做三件事，理解这三件事就能避免很多坑：

自动步骤	在哪定义	产物
用它自带的 glslc 编译 shader	CMakeLists.txt:67-109	out/build/.../shaders/*.spv

自动步骤	在哪定义	产物
运行反射生成器	CMakeLists.txt:112-135	tool/ReflectGenerated/generated/*.generated.cpp
把 assets/ 拷到 exe 旁边	CMakeLists.txt:207-211	<exe 目录>/assets/

程序通过 FileSystem::GetAssetsFolder() 从 exe 所在目录查找资源，不依赖当前工作目录（见 shared/file/FileSystem.cpp）。

三、按这个顺序读代码（从入口到细节，共 7 步）

第 1 步：入口，只看 main 的三行

src/main.cpp 只有 37 行：

代码块

```
SuzzaApplication* suzApp = new SuzzaApplication();
Engine::GetEngineInstance().SetApplication(suzApp);
engine.Init();  engine.Run();  engine.Destroy();
```

关键认知：引擎（Engine）和“游戏内容”（SuzzaApplication）是**分离的**。引擎只负责运行循环、Vulkan 和窗口；游戏内容由你自己写。这是整个项目最重要的一条分界线。

第 2 步：引擎的三大阶段 Init / Run / Destroy

看 src/engine/Engine.cpp:67、:106、:139，重点看 Init() 的**顺序**：

1. 注册所有反射类元数据（RegisterAllReflectionClass）
2. 创建图片和模型加载调度器
3. 创建窗口 MainWindow
4. InitVulkan() 初始化全部 Vulkan 资源
5. 创建 Render
6. 编译 EDITOR 宏时创建 EditorUI（该宏在非生产构建中默认开启，

见 CMakeLists.txt:28）

7. 最后才调用 m_suzzaApplication->Init()；**游戏对象在所有图形资源就绪后才创建**

Run() 就是主循环：glfwPollEvents → 编辑器 BeginFrame → 算 deltaTime → **App.Update** → **Render.Draw**。

第 3 步：你的"游戏代码"在哪：SuzzaApplication

src/SuzzaApplication.cpp（141 行）就是**最好的教程页面**，读完后应能回答："一个立方体是怎样被画到屏幕上的？"它做了：

1. 手写 24 个顶点 + 36 个索引 → new Mesh(...) → LoadMesh 得到字符串 ID
2. new Material() → LoadMaterial 得到 ID
3. 创建 Actor 树：RootActor → ChildA → GrandChild
4. 每个 Actor 挂一个 MeshComponent，调 CreateMesh(id)、CreateMaterial(id)
5. 再建一个 PlayerController，给它挂相机 CameraComponent

建议新手的第一件事就是修改这个文件：修改顶点坐标、修改 SetActorLocation，或者把注释里的模型加载打开（第 84-85 行 注释掉的 LoadModel 可以加载 assets/model/pony_cartoon/scene.gltf）。

第 4 步：场景与对象模型（src/scene/**）**

这是引擎最核心的架构，四个文件就够：

文件	职责	你要抓住的点
src/scene/Scene.h	场景容器	m_actorList（unique_ptr 拥有所有权）+ m_rootActorList；RegisterActor 处理"父为空、已有父、新对象"三种情况并做环检测
src/scene/Scene.cpp:165	场景更新	**只对根 Actor 调 Update**，子 Actor 由父递归调用；销毁采用"延迟删除"：先标记 PendingDestroy，本帧末尾执行 Destroy，下帧才 erase

文件	职责	你要抓住的点
src/scene/Actor.h	实体	组件数组 + 父子树 + 三件套 Transform（m_location / m_rotation 四元数 / m_scale）；世界矩阵 = 父世界矩阵 × 本地矩阵
src/scene/Component.h	组件基类	纯虚 BeginPlay/Update/Destroy；GetTypeId() 用模板静态计数器实现类型 ID；m_isAlive 是 shared_ptr<bool>，可跨对象判断有效性

Component 还有一个巧妙的宏：

```
#define COMPONENT(Comp) \
   static size_t GetComponentTypeId() { return GetStaticTypeId<Comp>(); } ...
```

配合 Actor::GetActorComponent<T>() 遍历 + 比较 typeId，实现了“按类型取组件”。

第 5 步：一帧是怎样画出来的（渲染主线）

把下面这条调用链背下来，Vulkan 部分就不再像黑盒：

代码块

```
Scene::Update
   └── MeshComponent::Update            (当前是空的)
Render::Draw                            [src/engine/model/Render.cpp:45]
   ├── vkWaitForFences   等上一帧
   ├── vkAcquireNextImageKHR   取交换链图像
   ├── BufferArea::RecordCommandBuffer     录制绘制命令
   |       ├── vkCmdBeginRenderPass
   |       ├── 遍历 mesh → vkCmdBindVertexBuffers / BindIndexBuffer / vkCmdDrawIndexed
   |       └── m_engineUIDelegate.Execute(cmd)    ← ImGui 编辑器在这里画
```

├── vkQueueSubmit

└── vkQueuePresentKHR [src/engine/vulkaninitialize/BufferArea.cpp:228-272]

关键理解：MeshComponent 并不直接画，它在 BeginPlay 阶段把 {Mesh*, ModelConstant{modelMatrix, slotIndex}} **提交到渲染队列**（src/scene/components/MeshComponent.cpp:95），Render 每帧再把队列中的内容写入 GPU。这种"提交队列 + 统一绘制"是引擎的常见做法。

第 6 步：Vulkan 层（可以最后看，也可以当教材看）

src/engine/vulkaninitialize/ 基本对应 Vulkan 教程的章节划分，一个类对应一个概念：

类	概念	备注
VulkanInitialize.h	Instance / Surface / 物理设备 / 逻辑设备 / 队列	还包含验证层、DescriptorIndexing 特性链（bindless 的前提）
SwapChain.h	交换链	窗口 resize 时重建
RenderPass.h	渲染通道	颜色 + 深度
Pipeline.h	图形管线	读取 shaders/ 下的 spv
Descriptor.h	描述符	相机 UBO + bindless 纹理数组
BufferArea.h	顶点/索引/UBO、帧缓冲、命令池、命令缓冲	文件名容易误解：它不只是"缓冲区"，而是"与交换链尺寸绑定的那一整套资源"
SyncObject.h	信号量 + 围栏	MAX_FRAMES_IN_FLIGHT = 2（见 VkMacro.h）

VulkanResourceManager 是这些类的统一容器（VulkanResourceManager.h），Engine 通过静态方法把它们暴露给全局。

第 7 步：资源管理与反射（进阶）

资源管理在 src/engine/model/：

- ModelResourceManager：用**随机字符串 ID** 缓存 Mesh/Material，并用 Mesh* → ID 的反向表防止重复加载
- BindlessSlotsAllocator：给每张纹理分配 0~255 的槽位（MAX_SAMPLED_TEXTURE_IMAGES_SLOTS），材质里存的就是这个 slot
- 模型加载和图片加载做成"管理器 + 调度器 + 具体实现"的三层结构，位于 tool/modelloader（cgltf）和 tool/imageloader（stb_image），方便以后接入新的格式

反射是最具特色、也最容易让新手困惑的部分，看懂三个文件就明白了：

1. 你在类里写 SUZZA_REFLECT_CLASS() 和 SUZZA_PROPERTY(Editability::Editable)（宏定义见 shared/macros/ReflectMacros.h）
2. 在 clang 下宏会展开为 __attribute__((annotate("property:", ...)))。**这只是给外部解析工具看的标记，MSVC 下展开为空**
3. 外部 LLVM 工具扫描源码，生成 Actor.generated.h/.cpp（已有产物例如 tool/ReflectGenerated/generated/Actor.generated.cpp），里面是 ClassInfo{ className, classPath, fields... }，每个字段带 getterFunc/setterFunc 函数对象
4. 编辑器 EditorUI::DrawInspectorPanel 遍历这些 FieldInfo，用 getter 取值、用 setter 写回，因此能在运行时修改属性

新手必知：*_REFLECT_BODY 宏使用 __LINE__ 拼接名字（Actor_REFLECT_CLASS_30_REFLECT_BODY），所以**反射类的行号一变，生成的代码就必须重新生成**。而生成器需要 LLVM 环境。现有 5 个类（Actor、CameraComponent、MeshComponent、MovementComponent、PlayerController）的生成代码已经提交在仓库里，直接修改数值没有问题；想加新反射类，要么安装 LLVM 重新生成，要么照着现成 .generated.h/.cpp 手动复制一份修改名字。

四、整体结构总览（新手版）

代码块

```
SuzzaEngine/
├── CMakeLists.txt                顶层构建：shader 编译、反射生成、资产拷贝
├── CMakePresets.json             x64-debug / x64-release 预设（Ninja +
```

cl.exe）

```
├── shaders/                    2 个 GLSL（顶点/片元），构建时编译成 spv
├── assets/                     模型(gltf/bin) 与纹理
│
├── src/                        ★ 引擎与游戏代码主体
│   ├── main.cpp                入口，37 行
│   ├── SuzzaApplication.*      ★ 游戏层：建 Actor、挂组件（新手从这里改）
│   │
│   ├── engine/                 ★ 引擎核心
│   │   ├── Engine.*            单例：Init / Run / Destroy，持有全部管理器
│   │   ├── MainWindow.*        GLFW 窗口、输入、resize 回调
│   │   ├── BaseObject.h        所有资源的基类（Destroy + IsAlive）
│   │   ├── input/              InputManager：按键/鼠标状态数组
│   │   ├── editor/             EditorUI：Hierarchy + Inspector
│   │   ├── model/              Mesh / Material / Texture / Render / ModelResourceManager
│   │   └── vulkaninitialize/   Vulkan 各概念一个类 + VulkanResourceManager
│   │
│   ├── scene/                  ★ ECS 架构
│   │   ├── Scene.*             场景、Actor 注册、延迟销毁、模型加载
│   │   ├── Actor.*             实体：Transform、组件表、父子树
│   │   ├── Component.*         组件基类 + 类型 ID 机制
│   │   ├── components/         Mesh / Camera / Movement 三个具体组件
│   │   └── actor/              PlayerController（一种 Actor 子类）
│   │
│   ├── reflection/             反射运行时：ClassInfo / FieldInfo /
```

ReflectionSystem

| ├── surface/ UI 抽象接口 IUIRender + ImGui 实现

| └── core/ 日志、数学等小工具（独立静态库 Suzza_Core_Moudle）

|

├── shared/ 跨目标共享：类型别名、文件系统、宏、数据结构

├── tool/

| ├── broadcast/ 自实现的单播/多播委托（C++ 信号槽）

| ├── imageloader/ stb_image 封装（管理器+调度器）

| ├── modelloader/ cgltf 封装

| └── ReflectGenerated/ ★ 反射生成器 exe + 已生成的代码

└── thirdparty/ imgui-docking / glfw / glm / cgltf / stb_image（裁剪过）

五、一帧的完整生命周期（读完后用它自检）

代码块

glfwPollEvents()

EditorUI::BeginFrame() // ImGui 建新帧

deltaTime = now - lastTime

SuzzaApplication::Update(deltaTime)

└── Scene::Update

└── 只有根 Actor 被调用 → Actor::Update 递归子 Actor

└── 每个 Component::Update // MovementComponent 在这里读输入、改 Transform

Render::Draw

├── 等 fence / 取图像 / 重建交换链(按需)

├── BufferArea::RecordCommandBuffer // mesh 队列 →

vkCmdDrawIndexed

│ └── ImGui 绘制命令（编辑器面板）

├── vkQueueSubmit → vkQueuePresentKHR

└── m_currentFrame = (m_currentFrame + 1) % 2

InputManager 更新鼠标上一帧坐标

自检问题:

1. 为什么 Scene::Update 里只对根 Actor 调用 Update?
2. 为什么 Mesh 数据要在 BeginPlay 提交，而不是在 Update?
3. m_currentFrame 为什么要对 2 取模?
4. 窗口拉伸时，哪些东西必须重建?

六、给新手的动手练习（由易到难）

难度	任务	涉及文件
★	把窗口标题/尺寸改成自己的	src/main.cpp
★	改立方体位置/缩放，观察父子 Actor 的联动	src/SuzzaApplication.cpp
★★	打开注释掉的模型加载，加载 pony_cartoon/scene.gltf	同上 第 84-85 行 + Scene.cpp 的 LoadModel
★★	写一个 RotateComponent：每帧给 Actor 加一点 Y 轴旋转	抄 MovementComponent.h 的写法
★★★	让 Inspector 面板显示并修改你新加的类型（如 float）	FieldInfo + EditorUI.cpp:72
★★★★	修改片元着色器，让物体按 UV 变色	shaders/simple_fragmentshader.frag

难度	任务	涉及文件
★★★★★	用 GetActorComponent<T>() 实现"相机跟随"	Actor.h 模板 + CameraComponent.h

---

七、新手极易踩的坑（先看，能省半天）

1. **源码是 GBK 编码**（注释是中文），不是 UTF-8。用 VS Code 打开会乱码，需设 files.encoding: gbk 或用 CLion 的 GBK 设置。若你新增文件用 UTF-8，MSVC 可能报 C4819。建议统一：要么全 GBK，要么全 UTF-8 BOM + /utf-8。
2. **新增反射类必须重新生成代码**，否则 #include "XXX.generated.h" 直接编译失败。手写生成文件是最快的兜底方案。
3. **资源路径**：运行时资源从 <exe 目录>/assets 读取，shader 从 <exe 目录>/shaders 读取。不要直接双击 out/build/.../SuzzaEngine.exe 之外的路径运行，也不要把 assets 拷贝错位置。
4. out/ **下有历史构建缓存**，如果编译报奇怪的错，删掉 out/build/x64-debug 重新 cmake --preset 最快。
5. **必须先 configure 再 build**，因为反射目标依赖 compile_commands.json（configure 阶段生成）。